

Practical-Week-8

Task-1

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    // malloc()
    int *arr = (int *)malloc(5 * sizeof(int));

    if (arr == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        arr[i] = i + 1;

    printf("Memory allocated using malloc(): ");
    for (i = 0; i < 5; i++)
        printf("%d ", arr[i]);
    printf("\n");

    // calloc()
    int *zero_arr = (int *)calloc(5, sizeof(int));

    if (zero_arr == NULL) {
        printf("calloc failed\n");
        free(arr);
        return 1;
    }

    printf("Memory allocated using calloc(): ");
    for (i = 0; i < 5; i++)
        printf("%d ", zero_arr[i]);
    printf("\n");

    // realloc()
    arr = (int *)realloc(arr, 10 * sizeof(int));

    if (arr == NULL) {
        if (zero_arr == NULL) {
            printf("calloc failed\n");
            free(arr);
            return 1;
        }

        printf("Memory allocated using calloc(): ");
        for (i = 0; i < 5; i++)
            printf("%d ", zero_arr[i]);
        printf("\n");

        // realloc()
        arr = (int *)realloc(arr, 10 * sizeof(int));

        if (arr == NULL) {
            printf("realloc failed\n");
            free(zero_arr);
            return 1;
        }

        for (i = 5; i < 10; i++)
            arr[i] = i + 1;

        printf("After realloc(): ");
        for (i = 0; i < 10; i++)
            printf("%d ", arr[i]);
        printf("\n");

        // free()
        free(arr);
        free(zero_arr);

        printf("Memory successfully released using free().\n");

        return 0;
    }
}
```

```

shivani@LAPTOP-N03KE600:~# mkdir memory_lab
cd memory_lab
nano memory_allocation.c
shivani@LAPTOP-N03KE600:~/memory_lab# nano memory_allocation.c
shivani@LAPTOP-N03KE600:~/memory_lab# gcc -Wall -g memory_allocation.c -o memory_allocation
shivani@LAPTOP-N03KE600:~/memory_lab# ./memory_allocation
Memory allocated using malloc(): 1 2 3 4 5
Memory allocated using calloc(): 0 0 0 0 0
After realloc(): 1 2 3 4 5 6 7 8 9 10
Memory successfully released using free().
shivani@LAPTOP-N03KE600:~/memory_lab# valgrind --version
valgrind-3.22.0
shivani@LAPTOP-N03KE600:~/memory_lab# valgrind --leak-check=full --show-leak-kinds=all ./memory_allocation
==1744== Memcheck, a memory error detector
==1744== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1744== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1744== Command: ./memory_allocation
==1744==
Memory allocated using malloc(): 1 2 3 4 5
Memory allocated using calloc(): 0 0 0 0 0
After realloc(): 1 2 3 4 5 6 7 8 9 10
Memory successfully released using free().
==1744==
==1744== HEAP SUMMARY:
==1744==   in use at exit: 0 bytes in 0 blocks
==1744==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==1744==
==1744== All heap blocks were freed -- no leaks are possible
==1744==
==1744== For lists of detected and suppressed errors, rerun with: -s
==1744== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
shivani@LAPTOP-N03KE600:~/memory_lab# |

```

Observation:

1. malloc(), calloc(), and realloc() successfully allocated and resized memory dynamically.
2. free() successfully released the allocated memory after program execution.
3. Valgrind reported **0 bytes in use at exit** and **0 errors**, indicating no memory leaks.
4. The memory allocation and deallocation behavior was successfully monitored using Valgrind.

Task-2:

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (10 * 1024 * 1024)

int main() {
    char *data = malloc(SIZE);

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    // Initialize the allocated memory
    for (size_t i = 0; i < SIZE; i += 4096) {
        data[i] = 'A';
    }

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated: 10 MB\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("\nChild PID: %d\n", getpid());
        printf("Before modification: %c\n", data[0]);

        sleep(2);

        // Modify memory in child
        for (size_t i = 0; i < SIZE; i += 4096) {
            data[i] = 'B';
        }

        printf("After modification: %c\n", data[0]);

        sleep(2);

        free(data);
        exit(0);
    }
    else {
        // Parent process
        printf("\nParent waiting...\n");

        sleep(5);

        printf("Parent data: %c\n", data[0]);

        wait(NULL);

        free(data);
        printf("Parent memory freed.\n");
    }

    return 0;
}
```

```
shivani@LAPTOP-N03KE600:~/memory_lab$ nano copy_on_write.c
shivani@LAPTOP-N03KE600:~/memory_lab$ gcc -Wall -g copy_on_write.c -o copy_on_write
shivani@LAPTOP-N03KE600:~/memory_lab$ ./copy_on_write
Parent PID: 1770
Memory allocated: 10 MB

Parent waiting...

Child PID: 1771
Before modification: A
After modification: B
Parent data: A
Parent memory freed.
shivani@LAPTOP-N03KE600:~/memory_lab$ |
```

Observation – 2nd Task: Copy-on-Write

1. After `fork()`, the parent and child initially share the same physical memory pages.
2. The child initially accesses the same data as the parent without creating a separate copy.
3. When the child modifies the data, the modified memory pages are copied for the child.
4. The parent retains the original data, confirming **Copy-on-Write behavior** and efficient memory usage.